

Autômatos de Pilha

Neste capítulo começamos a estudar a classe de autômatos que aceita as linguagens livres de contexto: os autômatos de pilha não determinísticos. Ao contrário dos autômatos finitos, os autômatos de pilha têm uma memória infinita. Contudo o acesso a esta memória é feito de maneira extremamente restrita, uma vez que o último item que foi posto na memória é obrigatoriamente o primeiro a ser consultado.

1. Heurística

Suponhamos que L é uma linguagem livre de contexto em um alfabeto Σ . Nesta seção consideramos como construir um procedimento que, tendo como entrada uma palavra w no alfabeto Σ , determina se w pertence ou não a L . Como sempre, assumiremos que w é lida, um símbolo de cada vez, da esquerda para a direita.

Nossos procedimentos utilizarão uma memória infinita, em forma de pilha. Para tornar o problema mais concreto, podemos imaginar que esta memória é constituída por discos perfurados ao meio, que são empilhados em uma haste. Os discos vêm em várias cores e temos um estoque infinito deles. Além disso, a haste pode ser feita tão longa quanto for necessário.

Para ‘lembrar’ alguma coisa, enfiamos discos coloridos na haste. Como os discos estão trespassados pela haste, só é possível removê-los um a um, começando sempre pelo que está mais acima.

Nosso primeiro exemplo é a linguagem L_1 formada pelas palavras no alfabeto $\{a, b, c\}$ que são da forma vcv^R , onde v é uma palavra qualquer nos as e bs . Por exemplo, as palavras $abaacaaba$ e $bbacabb$ pertencem a L_1 , mas $abcb$ e $abba$ não pertencem. É fácil mostrar que esta

linguagem não é regular usando o lema do bombeamento; portanto, não existe nenhum autômato finito que a aceite.

Para poder construir um procedimento que verifica se uma dada palavra de $\{a, b, c\}$ pertence ou não a L_1 precisamos ter uma maneira de ‘lembrar’ exatamente qual é a sequência de as e bs que apareceu antes do c . Comparamos, então, esta sequência com a que vem depois do c .

Faremos isto usando uma pilha e duas cores diferentes de discos: preto e branco. Procedemos da seguinte forma:

Etapas 1: Se achar a empilhe um disco preto na haste, e se achar b , um disco branco.

Etapas 2: Se achar c , mude de atitude e prepare-se para desempilhar discos.

Etapas 3: Compare o símbolo que está sendo lido com o que está no topo da pilha: se lê a e o topo da pilha é ocupado por um disco preto, ou se lê b e o topo é ocupado por um disco branco, desempilhe o disco.

Este procedimento obedece ainda a uma instrução que não foi explicitada acima, e que diz: se em alguma situação nenhuma das etapas acima puder ser aplicada, então páre de executar o procedimento.

Por exemplo, se a palavra dada for $w = abcba$, o procedimento se comporta da seguinte maneira:

Acha	Faz	Pilha	Resta na entrada
a	empilha disco preto	●	$bcba$
b	empilha disco branco	○ ●	cba
c	muda de atitude	○ ●	ba
b	compara e desempilha	●	a
a	compara e desempilha		

Observe que a pilha registra os símbolos de w que antecedem o c de baixo para cima. Os discos da pilha, por sua vez, são comparados, de cima para baixo, com a parte da palavra que sucede o c . Assim, o reflexo da parte da palavra que antecede o c é comparado com a parte da palavra que sucede o c . Portanto, se $w \in L_1$, então todos os símbolos de w devem ter sido consumidos e a pilha deve estar vazia quando o procedimento parar.

Por outro lado, se a palavra não está em L_1 então podem acontecer duas coisas. A primeira é que a entrada não possa ser totalmente

consumida por falta de instruções adequadas. Isto ocorre, por exemplo, quando a entrada é *cab*. A segunda possibilidade é que a entrada seja totalmente consumida, mas a pilha não se esvazie. Este é o caso, por exemplo, da entrada a^2bcba . Concluimos que a entrada deverá ser aceita se, e somente se, ao final da execução do procedimento ela foi totalmente consumida e a pilha está vazia.

A instrução de parar nos casos omissos faz com que o procedimento descrito acima seja completamente determinístico. Entretanto, nem sempre é possível criar um procedimento determinístico deste tipo para testar se uma palavra pertence a uma dada linguagem livre de contexto. Por exemplo, a linguagem

$$L_2 = \{vv^R : v \in \{a, b\}^*\},$$

no alfabeto $\{a, b\}$, é livre de contexto.

A única diferença desta linguagem para L_1 é que não há um símbolo especial marcando o meio da palavra. Portanto, se descobrirmos como identificar o meio da palavra, o resto do procedimento pode ser igual ao anterior. A saída é deixar por conta de quem está executando o procedimento o ônus de adivinhar onde está o meio da palavra. Como somente um símbolo da palavra é visto de cada vez, a decisão acaba tendo que ser aleatória. Portanto, tudo o que precisamos fazer é alterar a etapa 2, que passará a ser:

Nova etapa 2: decida (aleatoriamente) se quer mudar de atitude e passar a desempilhar os discos.

Observe que, se o procedimento determinar que a palavra dada pertence a L_2 , então podemos estar seguros de que isto é verdade. Entretanto, como o procedimento não é determinístico, uma saída negativa não garante que a palavra não pertence a L_2 . Podemos apenas ter sido infelizes na nossa escolha de onde estaria o meio da palavra.

2. Definição e exemplos

Vamos analisar os elementos utilizados na construção destes procedimentos e, a partir deles, sistematizar nossa definição de autômato de pilha.

Os elementos mais óbvios são: o alfabeto de entrada e a pilha. Entretanto, em ambos os exemplos temos mudanças de atitude de *empilha* para *compara e desempilha*. Como no caso de autômatos finitos, a atitude dos autômatos de pilha serão codificadas nos estados. Com isto precisamos também de um estado inicial que indica qual é a primeira

etapa do procedimento. Finalmente, precisamos de uma função de transição que nos diz o que fazer com a entrada (e a pilha!), e que seja flexível o suficiente para codificar procedimentos não determinísticos.

Comparando a análise acima com a definição de autômato finito, verificamos que não foi necessário mencionar estados finais. Afinal, a aceitação de uma entrada pelos nossos procedimentos foi determinada pelo fato da entrada ter sido totalmente consumida e pelo esvaziamento da pilha. Apesar disso, introduziremos a noção de estado final em nossa definição formal, porque isto nos dá maior flexibilidade à construção dos autômatos. Sistematizando estas considerações, obtemos a seguinte definição.

Um *autômato de pilha não determinístico* M fica completamente definido pelos seguintes elementos:

ALFABETO DE ENTRADA: Σ ;

ALFABETO DA PILHA: Γ ;

CONJUNTO DE ESTADOS: $Q = \{q_1, \dots, q_n\}$;

ESTADO INICIAL: q_1 ;

CONJUNTO DE ESTADOS FINAIS: $F \subseteq Q$;

FUNÇÃO DE TRANSIÇÃO: $\delta : Q \times (\Sigma \cup \{\epsilon\}) \times (\Gamma \cup \{\epsilon\}) \rightarrow \mathcal{P}_f(Q \times \Gamma^*)$.

Como no caso dos autômatos finitos, abreviaremos os elementos de um autômato finito listando-os em um vetor, sempre na ordem acima. Portanto, o autômato de pilha M que acabamos de definir tem como elementos $(\Sigma, \Gamma, Q, q_1, F, \delta)$.

Há algumas considerações que precisamos fazer sobre a função de transição definida acima. A primeira diz respeito ao seu conjunto de chegada. Se, por analogia com autômatos finitos não determinísticos, escolhêssemos este conjunto como sendo $\mathcal{P}(Q \times \Gamma^*)$, abriríamos a possibilidade de uma quantidade infinita de escolhas em uma transição. Isto porquê, ao contrário do que ocorria com autômatos finitos, o conjunto $Q \times \Gamma^*$ é infinito. Para evitar este problema, restringimos o conjunto de chegada a $\mathcal{P}_f(Q \times \Gamma^*)$, que é o conjunto formado pelos subconjuntos finitos de $Q \times \Gamma^*$.

Voltando nossa atenção agora para o conjunto de partida da função de transição, note que δ toma valores em triplas formadas por um estado, um símbolo do alfabeto de entrada e um símbolo do alfabeto da pilha. Não há nada de muito surpreendente até aí, porque estamos tentando modelar os procedimentos da seção 1, que consultam tanto a entrada quanto a pilha. Entretanto, diante destas considerações, esperaríamos

que o domínio de δ fosse $Q \times \Sigma \times \Gamma$, contudo, o que de fato obtemos é

$$Q \times (\Sigma \cup \{\epsilon\}) \times (\Gamma \cup \{\epsilon\}).$$

Isto significa que o autômato pode efetuar uma transição sem consultar a entrada ou sem consultar a pilha, ou ambos! Portanto podemos esperar destes autômatos um comportamento ainda ‘mais indeterminístico’ que o dos autômatos finitos.

Para poder enquadrar os procedimentos da seção 1 na estrutura da definição acima precisamos saber como interpretar o comportamento da função de transição em termos mais concretos. Digamos que M é um autômato de pilha não determinístico cujos elementos obedecem à notação adotada na definição de autômato de pilha não determinístico. Suponhamos que

$$q \in Q, \quad \sigma \in \Sigma \cup \{\epsilon\} \quad \text{e} \quad \gamma \in \Gamma \cup \{\epsilon\}.$$

Dados $p \in Q$ e $u \in \Gamma^*$, a condição

$$(p, u) \in \delta(q, \sigma, \gamma),$$

significa que se o autômato M está no estado q , lendo o símbolo σ na entrada, e tendo γ no topo da pilha, então ao consumir σ :

- M muda para o estado p ;
- M troca γ por u no topo da pilha.

Além disso, se $\sigma = \epsilon$ temos que a entrada não é consultada. Já quando $\gamma = \epsilon$, a transição é efetuada sem que o topo da pilha seja consultado.

Note que M pode trocar o símbolo do topo da pilha por uma palavra inteira. Em termos da pilha de discos perfurados da seção anterior isto significa que o disco do alto da pilha pode ser trocado por uma pilha de vários discos em uma única transição.

O fato de u poder ser uma palavra qualquer de Γ^* inclui a possibilidade de u ser ϵ . Mas o que significa trocar γ por ϵ no topo da pilha? Como ϵ não tem símbolos, retiramos γ e não pusemos nada no seu lugar. Portanto γ foi apenas removido da pilha.

Vale a pena enumerar os vários casos em que ϵ aparece para não deixar dúvidas quanto à interpretação correta de cada um:

Caso	Interpretação
$\sigma = \epsilon$	a entrada não é consultada
$\gamma = \epsilon$	o topo da pilha não é consultado
$\sigma = \gamma = \epsilon$	nem a entrada, nem o topo da pilha são consultados
$\gamma \neq \epsilon$ e $u = \epsilon$	remove γ da pilha
$\gamma = \epsilon$ e $u \neq \epsilon$	empilha u
$\gamma = \epsilon$ e $u = \epsilon$	não altera a pilha

O que vimos já é suficiente para que possamos adaptar os procedimentos da seção 1 ao modelo de autômato de pilha não determinístico.

Analisando o procedimento que aceita a linguagem L_1 , vemos que há apenas uma mudança de atitude, que corresponde à passagem de *empilha* para *desempilha*. Isto significa que o autômato de pilha $\mathcal{M}_1$ que desejamos construir deve ter dois estados, digamos q_1 e q_2 .

No primeiro estado, $\mathcal{M}_1$ põe um símbolo diferente na pilha para cada a ou b encontrado na entrada. Para facilitar, podemos imaginar que o alfabeto da pilha é $\{a, b\}$. Assim, para cada a achado na entrada, o autômato acrescenta um a no topo da pilha; e para cada b da entrada, um b é acrescentado ao topo da pilha. Temos assim as duas transições

$$\begin{aligned}\delta(q_1, a, \epsilon) &= \{(q_1, a)\}, \\ \delta(q_2, b, \epsilon) &= \{(q_2, b)\}.\end{aligned}$$

Note o ϵ indicando que um símbolo é empilhado, não importando o que esteja no topo da pilha.

Por outro lado, ao achar um c na entrada $\mathcal{M}_1$ muda de atitude, preparando-se para passar a desempilhar. Como esta mudança de atitude ocorre sem que nada seja feito à pilha, temos a transição

$$\delta(q_1, c, \epsilon) = \{(q_2, \epsilon)\}.$$

Já no estado q_2 , o autômato passa a comparar o símbolo da entrada com o que está no topo da pilha. Se os símbolos na entrada e na pilha coincidem, o autômato remove o símbolo que está no topo da pilha. Portanto,

$$\begin{aligned}\delta(q_2, a, a) &= \{(q_2, \epsilon)\}, \\ \delta(q_2, b, b) &= \{(q_2, \epsilon)\}.\end{aligned}$$

Para completar a descrição de $\mathcal{M}_1$ precisamos ainda decidir sobre seu estado inicial e seus estados finais. Não há problema quanto ao estado inicial que, claramente, deve ser q_1 . E quanto aos estados finais? De acordo com a definição do procedimento em 1, uma palavra é aceita

quando for inteiramente consumida e a pilha estiver vazia. Mas isso só pode acontecer quando o estado final for q_2 . Como é razoável esperar que, se o autômato aceita a entrada, então parou em um estado final, vamos declarar q_2 como sendo final.

A descrição do autômato acima seria, sem dúvida, mais fácil de interpretar se tivéssemos uma maneira mais compacta de descrever as transições. Como no caso dos autômatos finitos, faremos isto usando uma tabela que, além da palavra de entrada e do estado, registrará o que o autômato faz à pilha.

Suponhamos que temos um autômato de pilha e que $q, p_1, \dots, p_s$ são estados, σ é um símbolo do alfabeto de entrada, γ um símbolo do alfabeto da pilha e $u_1, \dots, u_s$ palavras no alfabeto da pilha. Uma transição

$$\delta(q, \sigma, \gamma) = \{(p_1, u_1), \dots, (p_s, u_s)\},$$

corresponderá a uma entrada da seguinte forma na tabela:

estado	entrada	topo da pilha	transições
q	σ	γ	(p_1, u_1) $\vdots$ (p_s, u_s)

Observe que os diferentes pares (estado, símbolo da pilha) de uma mesma transição são listados um sobre o outro sem uma linha divisória. As linhas horizontais da tabela separarão transições distintas uma da outra. Em muitos casos é conveniente acrescentar a este modelo básico de tabela uma quinta coluna com comentários sobre o que o autômato está fazendo naquela transição. É claro que estes comentários não fazem parte da descrição formal do autômato de pilha, eles apenas nos ajudam a entender como ele se comporta.

Utilizando esta notação, a tabela do autômato $\mathcal{M}_1$ é seguinte:

estado	entrada	topo da pilha	transições	comentários
q_1	a	ϵ	(q_1, a)	acha a e empilha a
q_1	b	ϵ	(q_1, b)	acha b e empilha b
q_1	c	ϵ	(q_2, ϵ)	acha c e muda de estado
q_2	a	a	(q_2, ϵ)	acha a na entrada e pilha e o desempilha
q_2	b	b	(q_2, ϵ)	acha b na entrada e pilha e o desempilha

No caso da linguagem L_2 da seção 1, o procedimento que criamos era não determinístico. Por isso, para modelá-lo como um autômato de pilha precisamos dar ao autômato a capacidade de, a qualquer momento, alterar o seu comportamento, deixando de empilhar e passando a desempilhar. Assim, enquanto está no estado q_1 o autômato pode ter duas atitudes:

Primeira: verificar qual é o símbolo de entrada e acrescentar o símbolo correspondente ao topo da pilha; ou

Segunda: ignorar a entrada e a pilha, mudar de estado e passar a desempilhar.

Levando em conta estas considerações, obtemos um autômato de pilha $\mathcal{M}_2$ que tem $\{a, b\}$ como alfabeto de entrada e da pilha; estados q_1 e q_2 ; estado inicial q_1 , estado final q_2 e função de transição dada por:

estado	entrada	topo da pilha	transições	comentários
q_1	a	ϵ	(q_1, a)	acha a e empilha a
q_1	b	ϵ	(q_1, b)	acha b e empilha b
q_1	ϵ	ϵ	(q_2, ϵ)	muda de estado
q_2	a	a	(q_2, ϵ)	acha a na entrada e pilha e o desempilha
q_2	b	b	(q_2, ϵ)	acha b na entrada e pilha e o desempilha

Observe que vale para a tabela a ressalva feita para os procedimentos na seção 1. Isto é, se surgir uma situação que leve a uma transição que não esteja especificada na tabela, então o autômato pára de se mover. Aliás, o mesmo valia para os autômatos finitos não determinísticos.

3. Computando e aceitando

Apesar de já termos uma descrição formal dos autômatos de pilha, ainda precisamos definir de maneira precisa o que significa um autômato de pilha aceitar uma linguagem. Faremos isto adaptando as noções correspondentes da teoria de autômatos finitos.

Seja M um autômato de pilha cujos elementos são dados pelo vetor $(\Sigma, \Gamma, Q, q_1, F, \delta)$. Uma *configuração* de M é um elemento de $Q \times \Sigma^* \times \Gamma^*$; ou seja, é uma tripla (q, w, u) onde

- q é um estado de M ;
- w é uma palavra no alfabeto de entrada;
- u é uma palavra no alfabeto da pilha.

Por uma questão de coerência com a maneira como a palavra w é lida, listamos os elementos da pilha em u de modo que o topo da pilha corresponda ao símbolo mais à esquerda de u .

Como ocorreu com os autômatos finitos, a finalidade desta definição é permitir que possamos acompanhar de maneira simples o comportamento de M com uma dada entrada. Portanto, esta definição só faz sentido quando vem conjugada à relação *configuração seguinte*. Suponhamos que $C = (q, \sigma w, \gamma u)$ é uma configuração de M , onde $\sigma \in \Sigma \cup \{\epsilon\}$, $\gamma \in \Gamma \cup \{\epsilon\}$. Dizemos que $C' = (q', w, vu)$ é uma das configurações seguintes a C se

$$(q', v) \in \delta(q, \sigma, \gamma).$$

Neste caso, escrevemos $C \vdash_M C'$.

Uma *computação* de M é uma seqüência de configurações $C_0, \dots, C_k$ tais que C_{i+1} é uma das configurações seguintes a C_i . Frequentemente abreviaremos a computação acima na forma $C_0 \vdash_M^k C_k$, já que, na maioria dos casos, estaremos interessados apenas nas configurações com as quais a computação começa e termina. Se o número de etapas da computação não for conhecido, ou não for relevante para as nossas considerações, escreveremos apenas $C_0 \vdash_M^* C_k$. Usaremos $\vdash$ em lugar de $\vdash_M$ a não ser que haja risco de confusão sobre qual o autômato que está sendo considerado.

Como no caso de autômatos finitos, convencionaremos que se C é uma configuração, então $C \vdash^0 C$, e também $C \vdash^* C$. Note, contudo, que $C \vdash^0 C$ não é equivalente a uma transição com entrada vazia. Por exemplo, o autômato $\mathcal{M}_2$ satisfaz

$$(q_1, \epsilon, \epsilon) \vdash (q_2, \epsilon, \epsilon).$$

Mas, apesar da entrada e da pilha não terem sido alteradas, as configurações inicial e final não coincidem.

Vejam estas definições em ação em alguns exemplos. Partindo da configuração $(q_1, abcba^2, a)$ temos a seguinte computação no autômato $\mathcal{M}_1$ da seção 2:

$$(q_1, abcba^2, a) \vdash (q_1, bcba^2, a^2) \vdash (q_1, cba^2, ba^2) \vdash (q_2, ba^2, ba^2) \vdash (q_2, a^2, a^2) \vdash (q_2, a, a) \vdash (q_2, \epsilon, \epsilon).$$

Esta é, essencialmente, a única computação possível partindo de $(q_1, abcba^2, a)$. A única coisa que podemos fazer para obter uma computação diferente é interromper a computação acima antes de chegar a $(q_2, \epsilon, \epsilon)$. Entretanto, isto está longe de ser sempre verdade, como mostra o exemplo seguinte.

Desta vez queremos considerar o autômato de pilha $\mathcal{M}_2$ definido ao final da seção 2. Digamos que a configuração inicial seja (q_1, a^2b, ϵ) . Uma computação possível é

$$(q_1, a^2b, \epsilon) \vdash (q_1, ab, a) \vdash (q_1, b, a^2) \vdash (q_2, b, a^2).$$

Mas há muitas outras possibilidades, como:

$$(q_1, a^2b, \epsilon) \vdash (q_2, a^2b, \epsilon)$$

ou ainda

$$(q_1, a^2b, \epsilon) \vdash (q_1, ab, a) \vdash (q_2, ab, a).$$

A diferença é que o primeiro autômato era basicamente determinístico, ao passo que este último é claramente não determinístico.

Supondo, como antes, que M é um autômato de pilha cujos elementos são dados pelo vetor $(\Sigma, \Gamma, Q, q_1, F, \delta)$, diremos que uma palavra $w \in \Sigma^*$ é *aceita* por M se existe uma computação

$$(q_1, w, \epsilon) \vdash^* (p, \epsilon, \epsilon),$$

onde p é um estado final de M . Como já ocorria no caso de autômatos finitos não determinísticos, basta que exista uma computação como acima para que a palavra seja aceita. A *linguagem* $L(M)$ *aceita por* M é o conjunto de todas as palavras que M aceita.

Em geral não é fácil determinar por simples inspecção qual a linguagem aceita por um autômato de pilha não determinístico dado. Contudo, veremos no próximo capítulo que é possível construir uma gramática livre de contexto que gere $L(M)$ diretamente da descrição de M .

Observe que três condições precisam ser *simultaneamente* satisfeitas para que possamos afirmar, ao final de uma computação, que M aceita w :

- (1) a palavra w tem que ter sido completamente consumida;
- (2) a pilha tem que estar vazia;
- (3) o autômato tem que ter atingido um estado final.

A condição referente ao estado final não aparece na descrição dos procedimentos na seção 1. De fato, ela surgiu na definição formal de autômato de pilha sob a pífia justificativa de que seria razoável exigir que o autômato parasse aceitando num estado final!

Como sempre acontece, a exigência de que o autômato tenha que atingir um estado final para que a entrada seja aceita simplifica a construção de alguns autômatos e complica a de outros. A verdade é que, como o exercício 7 mostra, teria sido possível suprimir toda menção a estados finais, embora isto não seja desejável do ponto de vista do desenvolvimento da teoria.

4. Variações em um tema

Nesta seção discutimos a construção de autômatos de pilha para três linguagens definidas de maneira muito semelhante. Com isto teremos a oportunidade de chamar a atenção para algumas dificuldades comuns; além de desenvolver técnicas simples, mas úteis, na construção de autômatos mais sofisticados.

Exemplo 1. Consideremos, em primeiro lugar, a linguagem livre de contexto

$$L = \{a^i b^i : i \geq 0\},$$

É fácil descrever um procedimento extremamente simples que usa uma pilha com apenas um tipo de disco para aceitar as palavras de L :

Etapa 1: Se achar um a na entrada, ponha um disco na pilha.

Etapa 2: Se achar um b na entrada mude de atitude e passe a comparar a entrada com a pilha, removendo um disco da pilha para cada b que achar na entrada (incluindo o primeiro!).

Para precisar o comportamento do autômato de maneira a não deixar dúvida sobre o que ele realmente faz basta construir sua tabela de transição.

estado	entrada	topo da pilha	transições	comentários
q_1	a	ϵ	(q_1, a)	empilha a
q_1	b	a	(q_2, ϵ)	desempilha a e muda de estado
q_2	b	a	(q_2, ϵ)	desempilha a

É claro que queremos que q_1 seja o estado inicial, mas precisamos tomar cuidado com a escolha dos estados finais. A primeira impressão talvez seja que q_2 é o único estado final. Entretanto, o autômato só pode

alcançar q_2 se houver algum símbolo na entrada, o que faria com que o autômato não aceitasse ϵ .

Há duas soluções possíveis. A primeira é declarar que q_1 também é um estado final. Isto com certeza faz com que ϵ seja aceita. Porém, precisamos nos certificar de que a inclusão de q_1 entre os estados finais não cria novas palavras aceitas que não pertencem a L . Fazemos isto analisando em detalhes o comportamento do autômato. Suponhamos, então, que $\epsilon \neq w \in \{a, b\}^*$ e que o autômato recebe w como entrada. Temos que:

- se w começa por b então nenhum de seus símbolos é consumido;
- se w começa por a então os a s são empilhados e para desempilhá-los é preciso chegar ao estado q_2 .

No primeiro caso a palavra não será aceita; no segundo, só será aceita se os a s são seguidos pelo mesmo número de b s e nada mais. Ou seja, se declaramos que os estados finais são $\{q_1, q_2\}$ então o autômato resultante aceita L .

A segunda possibilidade é alterar as transições e dar ao autômato a alternativa de, sem consultar a entrada ou a pilha, mudar de estado de q_1 para q_2 . Neste caso, o único estado final é $\{q_2\}$. Além disso, a transição que vamos acrescentar torna completamente redundante a terceira linha da tabela acima. A tabela que descreve a função de transição do autômato passa, então, a ser:

estado	entrada	topo da pilha	transições	comentários
q_1	ϵ	ϵ	(q_2, ϵ)	muda de estado
q_1	a	ϵ	(q_1, a)	empilha a
q_2	b	a	(q_2, ϵ)	desempilha a

O problema com esta última estratégia é que a transição que acrescentamos pode ser executada enquanto o autômato está no estado q_1 , não importando o que está sendo lido, nem o que há na pilha. Para ter certeza que tudo ocorre como esperado, devemos analisar o que o autômato faria se usasse esta transição “no momento errado”. Em primeiro lugar, se o autômato está no estado q_1 então ele não pode se mover ao ler b , a não ser que ignore a entrada e mude para o estado q_2 . Portanto, o autômato se comportará de maneira anômala apenas se ainda estiver lendo a e executar a transição que permite mudar de estado sem afetar a entrada nem a pilha. Neste caso ainda haveria a s a serem lidos. Como o autômato não se move se está no estado q_2 e lê a , então a computação

simplesmente pára. A conclusão é que o autômato continua aceitando o que devia apesar da alteração na tabela de transição.

Exemplo 2. A linguagem deste segundo exemplo é uma generalização da que aparece no primeiro. Seja L a linguagem formada pelas palavras $w \in \{a, b\}^*$ para as quais o número de as e bs é o mesmo. Assim, $abaaabbabb \in L$, mas $aaabaa \notin L$. Observe que a linguagem do exemplo anterior está contida em L .

À primeira vista, podemos construir um autômato de pilha que aceite L usando a mesma estratégia do Exemplo 1. Isto é, quando achamos a empilhamos a e quando achamos um b desempilhamos um a . O problema é o que fazer quando nos deparmos com uma palavra como $abba$. Neste caso começaríamos empilhando um a , mas em seguida teríamos de desempilhar dois as . Só que há apenas um a na pilha: como seria possível desempilhar mais as do que há na pilha?

A estratégia que vamos adotar consiste em construir um contador que:

- ao achar a soma 1 ao contador;
- ao achar b soma -1 ao contador.

Poderíamos fazer isto usando $\{-1, 1\}$ como alfabeto da pilha, entretanto, palavras como $-1 - 1$ estão demasiadamente sujeitas a causar confusão. Por isso, vamos simplesmente empilhar a para cada a encontrado na entrada, e b para cada b encontrado na entrada. Só precisamos lembrar que ‘empilhar um b sobre um a ’ tem o efeito de desempilhar o b da pilha, e vice-versa.

O que ocorre quando aplicamos esta estratégia à palavra $abba$? O primeiro a contribui um a para a pilha, que por sua vez é removido pelo b seguinte. Portanto, depois de ler o prefixo ab o autômato está com a pilha vazia. Em seguida aparece um b e o autômato põe um b na pilha. Este b é seguido por um a . Portanto teríamos que acrescentar um a no topo da pilha. Mas isto tem o efeito de desempilhar o b anterior, e a pilha acaba vazia, como desejado.

Parece fácil transcrever esta estratégia em uma tabela de transição, mas ainda há um obstáculo a vencer antes de podermos fazer isto. Afinal, para que esta estratégia funcione o autômato precisa ser capaz de detectar que a pilha está vazia. Entretanto, escrever ϵ para o topo da pilha em uma tabela de transição *não significa que a pilha está vazia*; significa apenas que o autômato não precisa saber o que está no topo da pilha ao realizar esta transição.

A saída é inventar um novo símbolo β para o alfabeto da pilha. Este símbolo é acrescentado à pilha ainda vazia, no início da computação.

Daí em diante, o autômato opera apenas com *as* e *bs* na pilha. Assim, ao avistar β no topo da pilha o autômato sabe que a pilha não contém mais nenhum *a* ou *b*. Para garantir que β só vai ser usado uma vez, convém reservar ao estado inicial apenas a ação de marcar o fundo da pilha. Portanto, se q_1 for o estado inicial teremos

$$\delta(q_1, \epsilon, \epsilon) = \{(q_2, \beta)\}.$$

Ao final desta transição, a entrada não foi alterada, o fundo da pilha foi marcado e o autômato saiu do estado q_1 para onde não vai poder mais voltar. O que o autômato deve fazer no estado q_2 está descrito resumidamente na seguinte tabela:

Acha na entrada	Acha na pilha	Faz
<i>a</i>	β ou <i>a</i>	empilha <i>a</i>
<i>b</i>	β ou <i>b</i>	empilha <i>b</i>
<i>b</i>	<i>a</i>	desempilha <i>a</i>
<i>a</i>	<i>b</i>	desempilha <i>b</i>

Com isto se, ao final da computação, a pilha contém apenas o marcador β então a palavra está em L e é aceita; do contrário a palavra é rejeitada. Entretanto, pela definição formal, dada na seção 3, o autômato só pode aceitar a palavra se a pilha estiver completamente vazia. Isto sugere que precisamos de mais uma transição para remover o β do fundo da pilha.

Com isto surge um novo problema. Se a palavra está em L , então será totalmente consumida deixando na pilha apenas β . Portanto, a transição que remove o β ao final desta computação, e esvazia completamente a pilha, não tem nenhuma entrada para consultar. O problema é que se permitimos ao autômato remover β sem consultar a entrada, ele pode realizar este movimento no momento errado, antes que a entrada tenha sido consumida. Como os nossos autômatos são não determinísticos, isto não apresenta nenhum problema conceitual.

Infelizmente o fato de ainda haver símbolos na entrada abre a possibilidade do autômato continuar a computação depois de ter retirado o marcador do fundo. Para evitar isto, o autômato deve, ao remover β , passar a um novo estado q_3 a partir do qual não há transições. Assim, se o autômato decidir remover o marcador antes da entrada ser totalmente consumida ele será obrigado a parar, e não poderá aceitar a entrada nesta computação. Como a pilha só vai poder se esvaziar em q_3 , é claro que este será o único estado final deste autômato.

Resumindo, temos um autômato que tem $\{a, b\}$ como alfabeto de entrada e da pilha; estados q_1 , q_2 e q_3 ; estado inicial q_1 ; estado final q_3 ; e cuja função de transição é definida na tabela 1.

estado	entrada	topo da pilha	transições	comentários
q_1	ϵ	ϵ	(q_2, β)	marca o fundo da pilha
q_2	a	β	$(q_2, a\beta)$	acha a e empilha um a
q_2	b	β	$(q_2, b\beta)$	acha b e empilha um b
q_2	a	a	(q_2, aa)	acha a e empilha um a
q_2	b	b	(q_2, bb)	acha b e empilha um b
q_2	b	a	(q_2, ϵ)	desempilha um a
q_2	a	b	(q_2, ϵ)	desempilha um b
q_2	ϵ	β	(q_3, ϵ)	esvazia a pilha

TABELA 1

Exemplo 3. A terceira linguagem que desejamos considerar é o conjunto das palavras $w \in \{a, b\}^*$ que têm mais as que bs . Vamos chamá-la de L_3 . À primeira vista, pode parecer que autômato é essencialmente igual ao anterior, bastando alterar a condição sob a qual a palavra é aceita. Afinal, se ao final da computação do autômato do Exemplo 2 sobram as na pilha, então a palavra de entrada tem mais as que bs . Mas surgem alguns problemas técnicos quando tentamos implementar esta estratégia.

A primeira dificuldade é que não temos como codificar nas transições o fato de não haver mais símbolos na entrada. A saída é permitir que o autômato tente advinhar isto por conta própria. Assim, ao achar um a na pilha, o autômato deve poder decidir que a computação chegou ao fim e, sem consultar a entrada, passar a um estado final. Naturalmente uma palavra não será aceita se a decisão de aplicar esta transição for tomada antes que a entrada tenha sido completamente consumida.

A segunda dificuldade é que estamos identificando que uma palavra está em L_3 porque sobram as na pilha. Entretanto, um dos requisitos para que uma palavra seja aceita por um autômato de pilha é que não sobrem símbolos na pilha ao final da computação! Resolvemos este conflito fazendo com que, ao chegar ao estado final, o autômato possa esvaziar a pilha sem se preocupar com a entrada.

Tomando por base o autômato do Exemplo 2, teremos que substituir a transição codificada na última linha da tabela, e acrescentar as três

transições que permitem ao autômato esvaziar a pilha. O autômato resultante terá alfabetos de entrada e da pilha iguais a $\{a, b\}$; estados q_1 , q_2 e q_3 ; estado inicial q_1 ; estado final q_3 ; e sua função de transição será dada na tabela 2.

estado	entrada	topo da pilha	transições	comentários
q_1	ϵ	ϵ	(q_2, β)	marca o fundo da pilha
q_2	a	β	$(q_2, a\beta)$	empilha um a
q_2	b	β	$(q_2, b\beta)$	empilha um b
q_2	a	a	(q_2, aa)	empilha um a
q_2	b	b	(q_2, bb)	empilha um b
q_2	b	a	(q_2, ϵ)	desempilha um a
q_2	a	b	(q_2, ϵ)	desempilha um b
q_2	ϵ	a	(q_3, a)	decide que a computação acabou com as sobrando na pilha
q_3	ϵ	a	(q_3, ϵ)	retira a da pilha
q_3	ϵ	b	(q_3, ϵ)	retira b da pilha
q_3	ϵ	β	(q_3, ϵ)	retira β da pilha

TABELA 2

5. Exercícios

1. Considere o autômato de pilha não determinístico $\mathcal{M}$ com alfabetos $\Sigma = \{a, b\}$ e $\Gamma = \{a\}$, estados q_1 e q_2 , estado inicial q_1 e final q_2 e transições dadas pela tabela:

estado	entrada	topo da pilha	transições
q_1	a	ϵ	(q_1, a) (q_2, ϵ)
q_1	b	ϵ	(q_1, a)
q_2	a	a	(q_2, ϵ)
q_2	b	a	(q_2, ϵ)

- (a) Descreva todas as possíveis seqüências de transições de $\mathcal{M}$ na entrada aba .
- (b) Mostre que aba , aa e abb não pertencem a $L(\mathcal{M})$ e que baa , bab e $baaaa$ pertencem a $L(\mathcal{M})$.

- (c) Descreva a linguagem aceita por $\mathcal{M}$ em português.
2. Ache um autômato de pilha não determinístico cuja linguagem aceita é L onde:
 - (a) $L = \{a^n b^{n+1} : n \geq 0\}$;
 - (b) $L = \{a^n b^{2n} : n \geq 0\}$;
 - (c) $L = \{w \in \{0, 1\}^* : w \text{ cujo número de } as \text{ é diferente do de } bs\}$;
 - (d) $L = \{a^n b^m : m, n \geq 0 \text{ e } m \neq n\}$;
 - (e) $L = \{w_1 c w_2 : w_1, w_2 \in \{a, b\}^* \text{ e } w_1 \neq w_2^r\}$;
 3. Um autômato finito não determinístico que aceita a linguagem denotada por $0 \cdot 0^* \cdot 1 \cdot 0$ não pode ter menos de 4 estados. Construa um autômato de pilha não determinístico com apenas 2 estados que aceita esta linguagem.
 4. Considere a linguagem dos parênteses balanceados descrita no exercício 2 da lista 4.
 - (a) Dê exemplo de uma gramática livre de contexto que gere esta linguagem.
 - (b) Dê exemplo de um autômato de pilha não determinístico que aceita esta linguagem.
 5. Esta questão trata da existência ou inexistência de computações infinitas.
 - (a) Explique porque um autômato finito (determinístico ou não) não admite uma computação com um número infinito de etapas.
 - (b) Dê exemplo de um autômato de pilha não determinístico que admite uma computação com um número infinito de etapas.
 6. Seja M um autômato de pilha. Mostre como definir a partir de M um novo autômato de pilha M_s que aceita a mesma linguagem que M e que, além disso, satisfaz às seguintes condições:
 - (a) a única transição de M_s a partir do seu estado inicial i é $\delta(i, \epsilon, \epsilon) = \{(q, \beta)\}$, onde q é um estado, β um símbolo do alfabeto da pilha e δ a função de transição de $\mathcal{M}_s$;
 - (b) o único estado de M_s a partir do qual há transições sem consultar a pilha é i ;
 - (c) M_s tem um único estado final f ;
 - (d) as transições que desempilham β levam o autômato ao estado f ;
 - (e) não há transições a partir de f .
 7. Construa um autômato de pilha não determinístico que aceite o complementar da linguagem

$$L = \{ww^R : w \in (0 \cup 1)^*\}.$$

8. Seja M um autômato de pilha não determinístico com alfabeto de entrada Σ , estado inicial q_1 e conjunto de estados Q . A linguagem que M aceita por pilha vazia é definida como:

$$N(M) = \{w \in \Sigma^* : \text{existe } (q_1, w, \epsilon) \vdash^* (q, \epsilon, \epsilon) \text{ onde } q \in Q\}.$$

Note que a diferença entre $L(M)$ e $N(M)$ é que, para a palavra ser aceita em $L(M)$ tem que ser possível chegar a uma configuração (q, ϵ, ϵ) em que q é um estado final, ao passo que não há restrições sobre o estado no caso de $N(M)$.

- Dê exemplo de um autômato de pilha não determinístico M para o qual $N(M) \neq L(M)$.
- Mostre que dado um autômato de pilha não determinístico M existe um autômato de pilha não determinístico M' tal que $L(M) = N(M')$.
- Mostre que dado um autômato de pilha não determinístico M existe um autômato de pilha não determinístico M' tal que $N(M) = L(M')$.

SUGESTÃO: use o autômato M_s construído no exercício anterior.

Resolução do Exercício 7

O autômato tem alfabeto de entrada $\{0, 1\}$, alfabeto da pilha $\{\beta, 0, 1\}$, conjunto de estados $\{q_1, \dots, q_4\}$, estado inicial q_1 , estado final q_4 e a seguinte tabela de transição:

estado	entrada	topo da pilha	transições	comentários
q_1	ϵ	ϵ	(q_2, β)	marca o fundo da pilha
q_2	0	ϵ	$(q_2, 0)$	empilha 0
q_2	1	ϵ	$(q_2, 1)$	empilha 1
q_2	ϵ	ϵ	$(q_3, 0)$	tenta advinhar o meio da entrada
q_3	0	0	(q_3, ϵ)	desempilha 0s casados
q_3	1	1	(q_3, ϵ)	desempilha 1s casados
q_3	0	1	(q_4, ϵ)	acha símbolo descasado
q_3	1	0	(q_4, ϵ)	acha símbolo descasado
q_4	0	0	(q_4, ϵ)	continua a desempilhar
q_4	0	1	(q_4, ϵ)	continua a desempilhar
q_4	1	0	(q_4, ϵ)	continua a desempilhar
q_4	1	1	(q_4, ϵ)	continua a desempilhar
q_4	ϵ	β	(q_4, ϵ)	esvazia a pilha

Depois de marcar o fundo da pilha o autômato empilha 0s e 1s até decidir, de maneira não determinística, que o meio da palavra de entrada foi encontrado. Então, muda de estado e passa a desempilhar em busca de um símbolo da pilha que não corresponda ao que está vindo na entrada. Ao achar um tal símbolo o autômato muda para o estado final e esvazia a pilha, comparando-a símbolo a símbolo com a entrada. Precisamos fazer isto para ter certeza de que o autômato advinhou o meio da palavra corretamente. Se não sobram nem 0s, nem 1s na pilha, o autômato remove o marcador do fundo e esvazia a pilha.

Note que, se o autômato advinhar incorretamente o meio da palavra, então sobrarão símbolos na entrada (se advinhar cedo demais) ou na pilha (se advinhar tarde demais). Em nenhum destes casos a palavra será aceita. Um destes casos ocorrerá obrigatoriamente se a palavra tiver comprimento ímpar. Finalmente, se a palavra da entrada estiver em L e se o autômato advinhar corretamente onde está o meio da palavra então a entrada será consumida e todos os 0s e 1s serão removidos da pilha. Entretanto, como o autômato continua no estado q_3 o marcador do fundo da pilha não é removido, de modo que a pilha não será esvaziada e a palavra não será aceita.

Gramáticas e autômatos de pilha

Nosso objetivo neste capítulo é dar uma demonstração do seguinte resultado fundamental.

TEOREMA 13.1. *Uma linguagem é livre de contexto se, e somente se, é aceita por algum autômato de pilha não determinístico.*

Como uma linguagem é livre de contexto se for gerada por uma gramática livre de contexto, o que precisamos fazer é estabelecer um elo entre gramáticas livres de contexto e autômatos de pilha.

Já temos alguma experiência com este tipo de questão. De fato, provamos que uma linguagem gerada por uma gramática linear à direita é regular construindo um autômato finito cujas computações simulam derivações na gramática dada. No caso de linguagens livres de contexto a correspondência será entre computações no autômato de pilha e derivações mais à esquerda na gramática.

Como no caso das linguagens regulares o problema pode ser dividido em dois. Assim, dada uma gramática livre de contexto $\mathcal{G}$ precisamos de uma receita para construir um autômato de pilha não determinístico que aceite $L(\mathcal{G})$. Reciprocamente, dado um autômato de pilha $\mathcal{M}$ queremos construir uma gramática livre de contexto que gere a linguagem aceita por $\mathcal{M}$.

1. O autômato de pilha de uma gramática

Seja $\mathcal{G}$ uma gramática livre de contexto. Nosso objetivo nesta seção consiste em construir um autômato de pilha cujas computações simulem as derivações à esquerda em $\mathcal{G}$. É claro que a pilha tem que desempenhar um papel fundamental nesta simulação. O que de fato acontece é que o

papel dos estados é secundário, e a simulação se dá na pilha. Dizendo de outra maneira, construiremos um autômato de pilha que simula *na pilha* as derivações mais à esquerda em $\mathcal{G}$.

Digamos que a gramática livre de contexto $\mathcal{G}$ é definida pela quádrupla de elementos (T, V, S, R) , e seja $\mathcal{M}$ o autômato de pilha a ser construído a partir de $\mathcal{G}$. Como $\mathcal{M}$ deve aceitar $L(\mathcal{G})$, é claro que seu alfabeto de entrada tem que ser T . Chamaremos a função de transição de $\mathcal{M}$ de δ e seu estado inicial de i .

A maneira mais concreta de pôr em prática a idéia delineada acima consiste em escolher o alfabeto da pilha como sendo $T \cup V$, e fazer corresponder a cada derivação de $\mathcal{G}$ uma transição do autômato. Entretanto, como a derivação de palavras de $L(\mathcal{G})$ é feita a partir do símbolo inicial S , o autômato deve começar pondo este símbolo no fundo da pilha. Observe que o autômato não pode consumir entrada ao marcar o fundo da pilha, já que precisamos da entrada para guiá-lo na busca da derivação correta. Nem adianta consultar a pilha, já que ainda está totalmente vazia. Entretanto, uma transição que não consulta a entrada nem a pilha pode ser executada em qualquer momento da computação. Por isso forçamos o autômato a mudar de estado depois desta transição, impedindo assim que volte a ser executada. Temos, então, que

$$\delta(i, \epsilon, \epsilon) = \{(f, S)\},$$

onde f é um estado do autômato diferente de i .

Daí em diante toda a ação vai se processar na pilha, e podemos simular a derivação sem nenhuma mudança de estado adicional. Portanto, f será o estado final de $\mathcal{M}$. Assim, à regra $X \rightarrow \alpha$ de $\mathcal{G}$ fazemos corresponder a transição

$$(1.1) \quad (f, \epsilon, \alpha) \in \delta(f, X)$$

de $\mathcal{M}$. Contudo, a construção do autômato ainda não está completa. O problema é que $\mathcal{M}$ só pode aplicar uma transição como (1.1) se a variável X estiver *no topo da pilha*. Infelizmente isto nem sempre acontece, como ilustra o exemplo a seguir.

Suponhamos que $\mathcal{G}_1$ é a gramática com terminais $\{a, b, c\}$, variável $\{S\}$, símbolo inicial S , e regras

$$S \rightarrow Sc \quad | aSb \quad | \quad \epsilon.$$

De acordo com a discussão acima o autômato $\mathcal{M}_1$ correspondente a $\mathcal{G}_1$ deve ter $\{a, b, c\}$ como alfabeto de entrada, $\{a, b, c, S\}$ como alfabeto da pilha e conjunto de estados $\{i, f\}$, onde i é o estado inicial e f o estado final. Lembrando que a cada regra de $\mathcal{G}_1$ deve corresponder uma transição de $\mathcal{M}_1$, concluímos que a tabela resultante deve ser

Estado	Entrada	Topo da pilha	Transição
i	ϵ	ϵ	(f, S)
f	ϵ	S	(f, Sc) (f, aSb) (f, ϵ)

A palavra abc^2 tem derivação mais à esquerda

$$S \Rightarrow Sc \Rightarrow Sc^2 \Rightarrow aSbc^2 \Rightarrow abc^2$$

em $\mathcal{G}_1$. Resta-nos verificar se somos capazes, ao menos neste exemplo, de construir uma computação de $\mathcal{M}_1$ que copie na pilha esta derivação de abc^2 . A computação deve começar marcando o fundo da pilha com S e deve prosseguir, a partir daí, executando, uma a uma, as transições de $\mathcal{M}_1$ que correspondem às regras aplicadas na derivação acima. Isto nos dá

(1.2)

$$(i, abc^2, \epsilon) \vdash (f, abc^2, S) \vdash (f, abc^2, Sc) \vdash (f, abc^2, Sc^2) \vdash (f, abc^2, aSbc^2).$$

Até aqui tudo bem, mas a transição seguinte deveria substituir S por ϵ . Só que para isto ser possível a variável S tem que estar no topo da pilha, o que não acontece neste caso. Observe, contudo, que o a que apareceu na pilha acima do S na última configuração de (1.2) corresponde ao primeiro símbolo da palavra de entrada. Além disso, como se trata de uma derivação mais à esquerda, este símbolo não será mais alterado. Concluímos que, como o primeiro símbolo da palavra já foi corretamente derivado, podemos ‘esquecê-lo’ e partir para o símbolo seguinte. Para implementar isto na prática basta apagar da pilha os terminais que precedem a variável mais à esquerda da pilha *e que já foram corretamente construídos*. Isto significa acrescentar à tabela acima transições que permitam apagar terminais que apareçam simultaneamente na entrada e na pilha:

Estado	Entrada	Topo da pilha	Transição
f	a	a	(f, ϵ)
f	b	b	(f, ϵ)
f	c	c	(f, ϵ)

Levando isto em conta a computação acima continua, a partir da última configuração de (1.2) da seguinte maneira

(1.3)

$$(f, abc^2, aSbc^2) \vdash (f, bc^2, Sbc^2) \vdash (f, bc^2, bc^2) \vdash (f, c^2, c^2) \vdash (f, c, c) \vdash (f, \epsilon, \epsilon).$$

Observe que a passagem da segunda para a terceira configuração em (1.3) corresponde à regra $S \rightarrow \epsilon$. Todas as outras etapas são aplicações das transições da segunda tabela.

2. A receita e mais um exemplo

Podemos agora descrever de maneira sistemática a receita usada para construir um autômato de pilha não determinístico $\mathcal{M}$ cuja linguagem aceita é $L(\mathcal{G})$. Se a gramática livre de contexto $\mathcal{G}$ tem por elementos (T, V, S, R) , então o autômato ficará completamente determinado pelos seguintes os elementos:

- o alfabeto de entrada T ;
- o alfabeto da pilha $T \cup V$;
- o conjunto de estados $\{i, f\}$;
- o estado inicial i ;
- o conjunto de estados finais $\{f\}$;
- a função de transição

$$\delta : \{i, f\} \times (T \cup \{\epsilon\}) \times (T \cup V \cup \{\epsilon\}) \rightarrow \{i, f\} \times (T \cup V)^*$$

que é definida por

$$\delta(q, \sigma, \gamma) = \begin{cases} (f, S) & \text{se } q = i \text{ e } \sigma = \gamma = \epsilon \\ \{(f, u) : \text{onde } X \rightarrow u \text{ é regra de } \mathcal{G}\} & \text{se } q = f \text{ e } \gamma = X \in V \\ (f, \epsilon) & \text{se } q = f \text{ e } \sigma = \gamma \in T \end{cases}$$

Este autômato executa dois tipos diferentes de transição a partir do estado f . As primeiras permitem substituir uma variável X no topo da pilha por $u \in (T \cup V)^*$ quando $X \rightarrow u$ é uma regra de $\mathcal{G}$, e serão chamadas *substituições*. As segundas permitem remover terminais que aparecem casados na pilha e na entrada, e vamos chamá-las *remoções*. Observe que, em geral, este autômato terá um comportamento muito pouco determinístico porque cada uma de suas transições corresponde ao conjunto de todas as regras que têm uma mesma variável do lado direito.

Vejamos mais um exemplo de autômato de pilha construído a partir de uma gramática livre de contexto pela receita acima. Considere a

gramática $\mathcal{G}'_{exp}$, definida no capítulo 9, cujas regras são

$$\begin{aligned} E &\rightarrow E + T \quad | \quad T \\ T &\rightarrow T * F \quad | \quad F \\ F &\rightarrow (E) \quad | \quad \text{id.} \end{aligned}$$

O alfabeto de entrada do autômato de pilha $\mathcal{M}'_{exp}$ correspondente a esta gramática é $\{id, +, *, (,)\}$, o alfabeto da pilha é $\{id, +, *, (,), E, T, F\}$, os estados são $\{i, f\}$, o estado inicial é i , o estado final é f e a função de transição é definida pela tabela 1.

Estado	Entrada	Topo da pilha	Transição
i	ϵ	ϵ	(f, S)
f	ϵ	E	$(f, E + T)$ (f, T)
f	ϵ	T	$(f, T * F)$ (f, F)
f	ϵ	F	$(f, (E))$ (f, id)
f	$($	$($	(f, ϵ)
f	$)$	$)$	(f, ϵ)
f	$+$	$+$	(f, ϵ)
f	$*$	$*$	(f, ϵ)
f	id	id	(f, ϵ)

TABELA 1. Tabela de transição

Quando criamos a receita acima, nosso objetivo era inventar um autômato que aceitasse a linguagem gerada por uma gramática livre de contexto dada. Mas para ter certeza de que a receita funciona precisamos provar o seguinte resultado.

Se $\mathcal{G}$ é uma gramática livre de contexto e $\mathcal{M}$ é o autômato de pilha não determinístico construído de acordo com a receita acima, então $L(\mathcal{G}) = L(\mathcal{M})$.

A demonstração deste resultado consiste em formalizar a relação entre derivações mais à esquerda em $\mathcal{G}$ e computações em $\mathcal{M}$, que foi o nosso ponto de partida para a criação da receita. Antes de fazer isto, porém, vale a pena tentar equiparar cada etapa de uma derivação mais à

esquerda em $\mathcal{G}'_{exp}$ com uma computação no autômato que acabamos de descrever. Por exemplo, vamos construir a computação correspondente à derivação mais à esquerda

$$E \Rightarrow E + T \Rightarrow T + T \Rightarrow F + T \Rightarrow \text{id} + T \Rightarrow \text{id} + F \Rightarrow \text{id} + \text{id}$$

e compará-las passo-a-passo. A derivação começa marcando o fundo da pilha com o símbolo inicial de $\mathcal{G}'_{exp}$

$$(i, \text{id} + \text{id}, \epsilon) \vdash (f, \text{id} + \text{id}, E),$$

mas isto não corresponde a nenhuma etapa da derivação. Em seguida, utilizando transições por substituição, reproduzimos na pilha as quatro primeiras etapas da derivação

$$(f, \text{id} + \text{id}, E) \vdash (f, \text{id} + \text{id}, E + T) \vdash (f, \text{id} + \text{id}, T + T) \vdash (f, \text{id} + \text{id}, F + T) \vdash (f, \text{id} + \text{id}, \text{id} + T).$$

Neste ponto nos deparamos com a necessidade de usar remoções para eliminar os terminais que obstruem o topo da pilha:

$$(f, \text{id} + \text{id}, \text{id} + T) \vdash (f, +\text{id}, +T) \vdash (f, \text{id}, T).$$

A computação prossegue com a aplicação de mais duas substituições e a remoção do último terminal na pilha e na entrada:

$$(f, \text{id}, T) \vdash (f, \text{id}, F) \vdash (f, \text{id}, \text{id}) \vdash (f, \epsilon, \epsilon).$$

Como era esperado, o número de passos na computação excede em muito o número de etapas da derivação por causa das transições de remoção, que não correspondem a nenhuma regra de $\mathcal{G}'_{exp}$. Observe que é muito fácil determinar se um passo da computação corresponde ou não a uma etapa da derivação; isto é, a uma transição por substituição. De fato, estas transições só podem ser aplicadas a configurações nas quais há uma variável no topo da pilha. Esta observação vai desempenhar um papel importante na demonstração de que nossa receita funciona.

3. Provando a receita

Como sempre a demonstração pode ser dividida em duas partes, que correspondem às inclusões $L(\mathcal{G}) \subseteq L(\mathcal{M})$ e $L(\mathcal{G}) \supseteq L(\mathcal{M})$.

Seja $\mathcal{G}$ é uma gramática livre de contexto formada pelos elementos (T, V, S, R) e seja w uma palavra gerada por $\mathcal{G}$. Queremos mostrar que w é aceita por $\mathcal{M}$. Suponhamos que conhecemos uma derivação mais à esquerda $S \Rightarrow^* w$ em $\mathcal{G}$. Devemos ser capazes de usar esta derivação para obter uma computação

$$(i, w, \epsilon) \vdash^* (f, \epsilon, \epsilon).$$

Além disso, sabemos que cada etapa desta computação deve corresponder a uma etapa da derivação, ou à aplicação de uma das transições por remoção, que apagam os prefixos de terminais que já tenham sido corretamente gerados na pilha de $\mathcal{M}$.

Digamos que, depois de k etapas, a derivação seja

$$S \Rightarrow^k \alpha X v,$$

onde estamos assumindo que $\alpha \in T^*$ e que $X \in V$. Em particular, X é a variável mais à esquerda de $\alpha X v$. Isto significa, por um lado, que w deve ser da forma $\alpha\beta$, para alguma palavra $\beta \in T^*$, e por outro que a próxima regra deve ser aplicada à variável X .

Se imaginarmos que já construímos a computação passo-a-passo até este ponto, devemos ter obtido

$$(i, w, \epsilon) \vdash (f, w, S) \vdash^* (f, \beta, X v),$$

porque os terminais de α foram sendo eliminados na mesma medida em que foram produzidos, para que a computação pudesse avançar.

Suponhamos que a regra aplicada na próxima etapa da derivação seja $X \rightarrow u$, onde $u \in (T \cup V)^*$. Avançando mais esta etapa na derivação, obtemos

$$S \Rightarrow^k \alpha X v \Rightarrow \alpha u v.$$

Queremos reproduzir esta etapa na computação de $\mathcal{M}$. Como já chegamos a um ponto em que X está no topo da pilha, basta aplicar a transição $(f, u) \in \delta(f, \epsilon, X)$, que nos dá

$$(i, w, \epsilon) \vdash (f, w, S) \vdash^* (f, \beta, X v) \vdash (f, \beta, uv).$$

Contudo isto não basta, porque queremos deixar a variável mais à esquerda de uv à descoberto, para poder aplicar a próxima regra sem obstáculos. Para isto precisamos localizar onde está esta variável, que vamos batizar de Y . Digamos que $uv = \gamma Y v'$, onde $\gamma \in T^*$. Como

$$\alpha uv = \alpha \gamma Y v' \Rightarrow w = \alpha \beta.$$

e γ só tem terminais, podemos concluir que γ é prefixo de β . Portanto, $\beta = \gamma\beta'$ e usando as regras de eliminação obtemos a computação

$$(i, w, \epsilon) \vdash^* (f, \beta, uv) = (f, \gamma\beta', \gamma Y v') \vdash^* (f, \beta', Y v'),$$

o que nos deixa prontos para prosseguir com a construção da computação exatamente como fizemos na etapa anterior.

Note que, continuando desta maneira, quando chegarmos ao final da derivação teremos obtido uma computação

$$(i, w, \epsilon) \vdash^* (f, \epsilon, \epsilon),$$

demonstrando, portanto, que $w \in L(\mathcal{M})$.

Provamos, assim, que $L(\mathcal{G}) \subseteq L(\mathcal{M})$, falta mostrar que a inclusão oposta também vale. Para isto, basta verificar que se w é aceita por $\mathcal{M}$ então existe uma derivação $S \Rightarrow^* w$.

A primeira impressão talvez seja de que, como a pilha de $\mathcal{M}$ reproduz uma derivação de w em $\mathcal{G}$, deve ser suficiente olhar para o que há na pilha em cada etapa da computação. Contudo, por causa das regras de remoção, cada vez que uma parte de w for gerada na pilha, ela será apagada antes da computação poder continuar. Portanto, para obter a etapa da derivação que corresponde a um dado passo da computação basta concatenar o prefixo de w que foi removido da entrada com o conteúdo da pilha. Em outras palavras, se a configuração de $\mathcal{M}$ em uma dada etapa da computação for (f, u, Yv) e $w = \alpha u$, então a etapa correspondente da computação será αYv .

O problema desta correspondência é que mais de uma etapa da computação pode corresponder a uma única etapa da derivação. Para evitar isto só vamos construir as etapas da derivação que correspondem a um passo da computação em que há uma variável no topo da pilha. Note que escolhemos estes passos porque é exatamente neles que se aplicam as transições por substituição.

Para concluir a demonstração precisamos nos convencer de que esta seqüência de palavras de $(T \cup V)^*$ é de fato uma derivação de w a partir de S . Há três coisas a verificar:

- (1) a seqüência começa com S ;
- (2) para passar de uma palavra da seqüência para a próxima trocamos sua variável mais à esquerda pelo lado direito de uma regra de $\mathcal{G}$;
- (3) a última palavra da seqüência é w .

Mas (1) e (2) são conseqüências imediatas da maneira como $\mathcal{M}$ foi definido, e (3) segue do fato de que a computação acaba na configuração (f, ϵ, ϵ) . Provamos, portanto, que $w \in L(\mathcal{G})$.

4. Autômatos de pilha cordatos

Na próxima seção consideramos a recíproca do problema descrito na seção anterior. Isto é, descrevemos um algoritmo que, a partir de um autômato de pilha $\mathcal{M}$ constrói uma gramática livre de contexto $\mathcal{G}$ tal que $L(\mathcal{G}) = L(\mathcal{M})$.

Para tornar a construção da gramática a partir do autômato mais fácil, começaremos transformando o autômato de pilha não determinístico $\mathcal{M}$ dado. Construiremos a partir de $\mathcal{M}$ um autômato de pilha $\mathcal{M}'$ que aceita a mesma linguagem que $\mathcal{M}$, mas cujo comportamento é mais

predizível. Em particular, $\mathcal{M}'$ terá apenas um estado final e a pilha só poderá se esvaziar neste estado.

Para melhor sistematizar os algoritmos é conveniente introduzir a noção de um autômato de pilha *cordato*. Seja $\mathcal{N}$ um autômato de pilha com estado inicial i e função de transição δ . Dizemos que $\mathcal{N}$ é *cordato* se as seguintes condições são satisfeitas:

- (1) a única transição a partir de i é $\delta(i, \epsilon, \epsilon) = \{(q, \beta)\}$, onde q é um estado de $\mathcal{N}$ e β é um símbolo do alfabeto da pilha;
- (2) $\mathcal{N}$ tem um único estado final f ;
- (3) a pilha só se esvazia no estado final f ;
- (4) não há transições a partir de f .

Note que por causa de (1), (3) e (4), há um elemento $\beta \in \Gamma$ que nunca sai do fundo da pilha até que o estado final seja atingido. Além disto, (3) nos diz que se $\delta(q, \sigma, \beta) = (p, \epsilon)$ então $p = f$.

Vejamos como é possível construir a partir de um autômato de pilha $\mathcal{M}$ qualquer um autômato de pilha cordato $\mathcal{N}$ que aceite $L(\mathcal{M})$. Suponhamos que $\mathcal{M}$ é definido pelos ingredientes $(\Sigma, \Gamma, Q, q_1, F, \delta)$. Então $\mathcal{N}$ será o autômato de pilha com os seguintes ingredientes:

- alfabeto de entrada Σ ;
- alfabeto da pilha $\Gamma \cup \{\beta\}$, onde $\beta \notin \Gamma$;
- conjunto de estados $Q \cup \{i, f\}$, onde $i, f \notin Q$;
- estado inicial i ;
- conjunto de estados finais $\{f\}$;
- função de transição δ' definida de acordo com a tabela abaixo:

estado	entrada	topo da pilha	transição
i	ϵ	ϵ	(q_1, β)
$q \neq i, f$ e $q \notin F$	σ	γ	$\delta(q, \sigma, \gamma)$
$q \neq i, f$ e $q \in F$	σ	$\gamma \neq \epsilon$	$\delta(q, \sigma, \gamma)$
$q \neq i, f$ e $q \in F$	ϵ	β	$\delta(q, \epsilon, \epsilon) \cup \{(f, \epsilon)\}$

onde estamos supondo que $\sigma \in \Sigma \cup \{\epsilon\}$ e que $\gamma \in \Gamma \cup \{\epsilon\}$.

É claro que $\mathcal{N}$ é cordato; falta apenas mostrar que $L(\mathcal{N}) = L(\mathcal{M})$. Observe que o comportamento de $\mathcal{N}$ pode ser descrito sucintamente dizendo que, depois de marcar o fundo da pilha com β , o autômato simula o comportamento de $\mathcal{M}$. De fato, na primeira transição $\mathcal{N}$ apenas põe β no fundo da pilha e passa ao estado q_1 de $\mathcal{M}$. A partir daí $\mathcal{N}$ se comporta como $\mathcal{M}$ até que um estado final p de $\mathcal{M}$ é atingido. Neste caso, se a pilha contém apenas β , $\mathcal{N}$ tem a opção de entrar no estado f

e esvaziar a pilha. Observe que a pilha só é esvaziada no estado f , e isto obriga o autômato a parar porque não existem transições a partir de f .

Suponha agora que $\mathcal{N}$ computa a partir de uma entrada $w \in \Sigma^*$. Se w for aceita por $\mathcal{M}$, então existirá uma computação

$$(q_1, w, \epsilon) \vdash^* (p, \epsilon, \epsilon) \text{ em } \mathcal{M}.$$

Esta computação dará lugar a uma computação em $\mathcal{N}$ da forma

$$(i, w, \epsilon) \vdash (q_1, w, \beta) \vdash^* (p, \epsilon, \beta) \vdash (f, \epsilon, \epsilon).$$

Portanto se w for aceita por $\mathcal{M}$ será aceita por $\mathcal{N}$.

Por outro lado, uma computação

$$(4.1) \quad (i, w, \epsilon) \vdash^* (f, \epsilon, \epsilon),$$

implica que a configuração do autômato $\mathcal{N}$ anterior à última tinha que ser (p, ϵ, β) , com p um estado final de $\mathcal{M}$. Assim, a computação (4.1) de $\mathcal{N}$ procede ao longo das seguintes etapas

$$(i, w, \epsilon) \vdash (q_1, w, \epsilon) \vdash^* (p, \epsilon, \beta) \vdash (f, \epsilon, \epsilon).$$

Temos assim que $(q_1, w, \epsilon) \vdash^* (p, \epsilon, \epsilon)$ é uma computação válida de $\mathcal{M}$. Como p é estado final de $\mathcal{M}$ concluímos que w é aceita por $\mathcal{M}$. Portanto $\mathcal{M}$ e $\mathcal{N}$ aceitam exatamente as mesmas palavras.

5. A gramática de um autômato de pilha

Suponha agora que $\mathcal{N}$ é um autômato de pilha cordato. Veremos como é possível construir uma gramática livre de contexto G que gera as palavras aceitas por $\mathcal{N}$. Digamos que $\mathcal{N}$ está definido pelos ingredientes $(\Sigma, \Gamma, Q, i, \{f\}, \delta)$. Além disso, suporemos que β é o símbolo que fica no fundo da pilha de $\mathcal{N}$ enquanto ele simula $\mathcal{M}$.

Naturalmente os terminais da gramática G serão os elementos de Σ . A construção das variáveis é mais complicada. Cada variável será indexada por três símbolos: dois estados e um símbolo da pilha. Como isto complica muito a notação é preferível identificar cada variável com a própria tripla que lhe serve de índice. Assim o conjunto de variáveis de G será

$$Q \times \Gamma \times Q.$$

Portanto uma variável de G será uma tripla (q, γ, p) , onde q e p são estados de $\mathcal{N}$ e γ é um símbolo do alfabeto da pilha. Note que γ não pode ser ϵ .

Se

$$\delta(i, \epsilon, \epsilon) = \{(i', \beta)\},$$

então o símbolo inicial de G será (i', β, f) . Precisamos agora construir as regras de G . Suponhamos que

$$(p, u) \in \delta(q, \sigma, \gamma)$$

onde $u \in \Gamma^*$. Há dois casos a considerar:

Primeiro caso: $u = \gamma_1 \cdots \gamma_k$.

Neste caso, para cada k -upla $(r_1, \dots, r_k) \in Q^k$ contruímos uma regra

$$(q, \gamma, r_k) \rightarrow \sigma(p, \gamma_1, r_1)(r_1, \gamma_2, r_2) \cdots (r_{k-1}, \gamma_k, r_k).$$

Observe que isto nos dá, não apenas uma, mas n^k regras distintas, onde n é o número de estados de $\mathcal{N}$.

Segundo caso: $u = \epsilon$

Neste caso construímos apenas a regra

$$(q, \gamma, p) \rightarrow \sigma.$$

De nossa experiência anterior sabemos que, de alguma maneira, uma derivação mais à esquerda por G deve simular uma computação por $\mathcal{N}$. Mais precisamente,

(5.1)

$$(i, w, \epsilon) \vdash (i', w, \beta) \vdash^* (f, \epsilon, \epsilon) \text{ se, e somente se } (i', \beta, f) \Rightarrow^* w.$$

Entretanto, neste caso a simulação procede de maneira bem mais sutil. Para melhor identificar o problema, consideremos uma etapa da computação acima

$$(q, \sigma v, \gamma u) \vdash (p, w, \gamma_1 \cdots \gamma_k u),$$

onde $v \in \Sigma^*$ e $u \in \Gamma^*$. Evidentemente para que esta etapa seja legítima é preciso que

$$(5.2) \quad (p, \gamma_1 \cdots \gamma_k) \in \delta(q, \sigma, \gamma).$$

Mas (5.2) dá lugar a n^k regras: qual delas devemos escolher? Para decidir isto precisamos verificar como a computação continua.

Em outras palavras, para construir o i -ésimo passo da derivação não é suficiente considerar apenas o i -ésimo passo da computação, mas vários outros passos; até mesmo todo o restante da computação! É claro que isto torna a demonstração da correspondência bem mais difícil que no caso de autômatos finitos. Felizmente podemos generalizar a equivalência (5.1) de modo a tornar a demonstração mais transparente. Para isso substituímos em (5.1) os estados i' e f por estados quaisquer p e q , e β por um símbolo qualquer X do alfabeto da pilha. Esta generalização é o conteúdo do seguinte lema.

LEMA 13.2. *Sejam p e q estados do autômato de pilha não determinístico $\mathcal{N}$. Então*

$$(q, w, X) \vdash^* (p, \epsilon, \epsilon) \text{ se, e somente se } (q, X, p) \Rightarrow^* w$$

na gramática $G(\mathcal{N})$.

De acordo com esta correspondência se o autômato está no estado p quando X é desempilhado, então a variável que inicia a derivação é (q, X, p) . Assim, para achar a variável da partida é preciso olhar a computação até o último passo!

A demonstração é por indução finita, e para torná-la mais clara vamos dividi-la em duas partes.

Primeira parte: Queremos mostrar a afirmação

$$(\mathbf{A}(m)) \quad \text{se } (q, w, X) \vdash^m (p, \epsilon, \epsilon), \text{ então } (q, X, p) \Rightarrow^* w.$$

por indução em m .

Se $m = 1$, então

$$w = \sigma \in \Sigma \cup \{\epsilon\} \text{ e } (p, \epsilon) \in \delta(q, \sigma, X).$$

Mas, por construção, isto significa que na gramática $G(\mathcal{N})$ há uma regra do tipo

$$(q, X, p) \rightarrow \sigma.$$

Como, neste caso, toda a derivação se reduz a uma aplicação desta regra, a base da indução está provada.

Suponhamos, agora, que $s > 1$ é um número inteiro, e que $\mathbf{A}(m)$ vale para todo $m < s$. Seja

$$(5.3) \quad (q, w, X) \vdash^s (p, \epsilon, \epsilon)$$

uma computação em $\mathcal{N}$ com s etapas.

Isolando o primeiro símbolo de w , podemos escrever $w = \sigma v$, onde $\sigma \in \Sigma$ e $v \in \Sigma^*$. Neste caso o primeiro passo da computação (5.3) será da forma

$$(q, \sigma v, X) \vdash (q^1, v, Y_1 \cdots Y_k),$$

que corresponde à transição

$$(5.4) \quad (q^1, v, Y_1 \cdots Y_k) \in \delta(q, \sigma, X).$$

Note que, ao final da computação (5.3) cada Y foi desempilhado. Lembre-se que dizer, por exemplo, que Y_1 é removido da pilha significa que a pilha passou a ser $Y_2 \cdots Y_k$. Isto não tem que acontecer em apenas uma transição. Assim, Y_1 pode ser trocado por uma palavra no alfabeto da pilha sem que seja necessariamente desempilhado. Por isso, durante os passos seguintes da computação a pilha pode crescer bastante antes

de diminuir ao ponto de ser apenas $Y_2 \cdots Y_k$. Digamos então que v_1 é o prefixo de v que é consumido para que Y_1 seja desempilhado. Se $v = v_1 v$, temos uma computação

$$(q^1, v_1 v', Y_1 \cdots Y_k) \vdash^* (q^2, v', Y_2 \cdots Y_k),$$

onde q^2 é algum estado de $\mathcal{N}$.

Analogamente, sejam $v_2, v_3, \dots, v_k$ as subpalavras de v que têm que ser consumidas para que $Y_2, \dots, Y_k$ sejam desempilhados. Assim, $v = v_1 v_2 \dots v_k$, e existem estados $q^1, \dots, q^k$ de $\mathcal{N}$ modo que (5.3) pode ser subdividida em $k - 1$ etapas da forma

$$(q^i, v_i \cdots v_k, Y_i \cdots Y_k) \vdash^* (q^{i+1}, v_{i+1} \cdots v_k, Y_{i+1} \cdots Y_k),$$

seguidas de uma etapa final da forma

$$(q^k, v_k, Y_k) \vdash^* (p, \epsilon, \epsilon).$$

Como cada uma destas computações tem menos de s passos, segue da hipótese de indução, que

$$(q^i, v_i, Y_i) \vdash^* (q^{i+1}, \epsilon, \epsilon),$$

dá lugar à derivação

$$(5.5) \quad (q^i, Y_i, q^{i+1}) \Rightarrow^* v_i,$$

ao passo que a etapa final da computação corresponde a

$$(5.6) \quad (q^k, Y_k, p) \Rightarrow^* v_k.$$

Falta-nos apenas reunir de modo ordenado tudo o que fizemos até agora. Em primeiro lugar, a transição (5.4) dá lugar a uma regra de $G(\mathcal{N})$ da forma

$$(q, X, p) \rightarrow \sigma(q^1, Y_1, q^2) \cdots (q^k, Y_k, p).$$

Substituindo, finalmente, as derivações de (5.5) e (5.6) nos lugares apropriados, obtemos

$$(q, X, p) \Rightarrow \sigma(q^1, Y_1, q^2) \cdots (q^k, Y_k, p) \Rightarrow^* \sigma v_1 \cdots v_k = w,$$

como queríamos.

Segunda parte: Queremos mostrar a afirmação

$$(B(m)) \quad \text{se } (q, X, p) \Rightarrow^m w \text{ então } (q, w, X) \vdash^* (p, \epsilon, \epsilon)$$

por indução em m

Se $m = 1$ então a derivação se resume a à regra

$$(q, X, p) \rightarrow w,$$

da gramática $G(\mathcal{N})$. Mas, pela construção da gramática, w tem que ser um símbolo $\sigma \in \Sigma \cup \{\epsilon\}$. Além disto, esta regra só pode ocorrer se houver uma transição da forma

$$(p, \epsilon) \in \delta(q, \sigma, X)$$

em $\mathcal{N}$. Mas esta transição dá lugar à computação

$$(q, \sigma, X) \vdash (p, \epsilon, \epsilon)$$

que esperávamos obter.

Suponha, agora, que $s > 1$ é um número inteiro e que o resultado vale para toda derivação com menos de s passos. Seja

$$(5.7) \quad (q, X, p) \Rightarrow^s w$$

uma derivação de $G(\mathcal{N})$ com s etapas. Como $s > 1$, a primeira regra a ser aplicada nesta derivação deve ser da forma

$$(5.8) \quad (q, X, p) \rightarrow \sigma(s_1, Y_1, s_2)(s_2, Y_2, s_3) \cdots (s_k, Y_k, p),$$

onde $(s_1, \dots, s_k) \in Q^k$ e σ é o símbolo mais à esquerda de w . Digamos que $w = \sigma v$, para alguma palavra $v \in \Sigma^*$. Para que a derivação possa acabar produzindo w deve ser possível decompor v na forma $v = v_1 \cdots v_k$ de modo que, para $i = 1, \dots, k$, temos

$$(s_i, Y_i, s_{i+1}) \Rightarrow^* v_i$$

onde $s_{k+1} = p$. Como a derivação de v_i s tem que ter um número de etapas menor que s , podemos aplicar a hipótese de indução, obtendo assim computações

$$(s_i, v_i, Y_i) \vdash^* (s_{i+1}, \epsilon).$$

para $i = 1, \dots, k$. Além disso, se pusermos $Y_{i+1}, \dots, Y_k$ abaixo de Y_i , no fundo da pilha, obteremos

$$(s_i, v_i, Y_i Y_{i+1} \cdots Y_k) \vdash^* (s_{i+1}, \epsilon).$$

Agora, (5.8) provém da transição

$$(p, Y_1 \cdots Y_k) \in \delta(q, \sigma, X),$$

que pode ser reescrita na forma

$$(q, \sigma, X) \vdash (p, Y_1 \cdots Y_k).$$

Lembrando que $w = \sigma v_1 \cdots v_k$, vemos que $\mathcal{N}$ admite uma computação da forma

$$(q, w, X) \vdash (s_1, v_1 \cdots v_k, Y_1 \cdots Y_k) \vdash^* (s_2, v_2 \cdots v_k, Y_2 \cdots Y_k) \vdash^* \cdots \vdash^* (s_k, v_k, Y_k) \vdash (p, \epsilon, \epsilon),$$

como queríamos mostrar.

6. Exercícios

1. Construa a computação no autômato descrito na seção 2 que corresponde à derivação mais à esquerda de $\text{id} * (\text{id} + \text{id})$ na gramática $\mathcal{G}'_{exp}$.
2. Ache um autômato de pilha não determinístico que aceita a linguagem gerada pela gramática cujas regras são:

$$S \rightarrow 0AA$$

$$A \rightarrow 1S|0S|0$$

3. Considere a linguagem dos parênteses balanceados descrita no exercício 2 do capítulo ??.

 - (1) Dê exemplo de uma gramática livre de contexto que gere esta linguagem.
 - (2) Dê exemplo de um autômato de pilha não determinístico que aceita esta linguagem.

4. Para cada uma das linguagens L , abaixo, invente uma gramática livre de contexto que gere L e use a receita da seção 2 para construir um autômato de pilha não determinístico que aceite L .
 - (a) $L = \{wc^4w^r : w \in \{0, 1\}^*\}$;
 - (b) $L = \{a^n b^m c : n \geq m \geq 1\}$;
 - (c) $L = \{0^m 1^n : n \leq m \leq 2n\}$;
 - (d) $L = \{a^{i+3} b^{2i+1} : i \geq 0\}$;
 - (e) $L = \{a^i b^j c^j d^i e^3 : i, j \geq 0\}$.- 5. Seja $\mathcal{G}$ uma gramática livre de contexto cujo símbolo inicial é S , e seja $\mathcal{M}$ o autômato de pilha construído a partir de $\mathcal{G}$ pela receita da seção 2. Suponhamos que w é uma palavra de comprimento k em $L(\mathcal{G})$. Determine o número de passos da computação de $\mathcal{M}$ que corresponde à derivação mais à esquerda $S \Rightarrow^n w$.